Autonomous Predictive Drip Irrigation and Water Consumption Monitoring System Using Hall Effect Sensor

Harsh Singh
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
harsh.singh2023c@vitstudent.ac.in

Manya Shrivastava
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
manya.shrivastava2023@vitstudent.ac.in

A. Charanjith Reddy
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
charanjithreddy.a2023@vitstudent.ac.in

Naresh K
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
naresh.k@vit.ac.in

***Abstract*—Smallholder drip irrigation can be made more water-efficient by combining local soil-moisture sensing with a short-term rainfall forecast, so that irrigation is withheld when rain is already expected. This paper reports the design and bench-prototype implementation of such a system, built around an ESP32 edge controller, a capacitive soil-moisture sensor, a DHT11 temperature/humidity sensor, a Hall-effect turbine flow sensor for volumetric water accounting, and a relay-driven pump/solenoid actuator. The firmware combines a local soil-moisture threshold with a direct HTTP call to an external weather-forecast API and withholds irrigation when dry soil and a forecast rain probability above a configurable threshold coincide. Two independent safety interlocks, a maximum pump run-time and a no-flow-pulse fault, force the actuator off regardless of the irrigation decision. The paper reports implementation status rather than performance figures: the firmware source, a bench serial-monitor log and photographs of the assembled hardware support the design, while calibration accuracy, water savings, forecast-skip precision and relay polarity remain to be validated. Each claim is classified by the strength of its supporting evidence using a five-category scheme that can be reused in other early-stage hardware projects.**

***Index Terms*—predictive irrigation, soil moisture sensing, ESP32, Hall-effect flow sensor, IoT agriculture, evidentiary classification, prototype implementation**

I. Introduction

The majority of freshwater withdrawn from India's water resources comes from the agricultural sector; national water management assessments have repeatedly shown that both groundwater over-pumping and inefficient irrigation are major structural threats to India's long-term food and water security [10], [11]. The continued decline in groundwater levels across northern India, as shown by satellite gravimetry data, further supports this finding [12]. Irrigation inefficiencies are primarily caused by a lack of predictive decision-making using weather forecasts, rather than a lack of sensors. Irrigation schedules are typically based upon the current state of the soil and do not consider possible rainfall events, which leads to unnecessary water use and pumping energy. Due to the availability of low-cost system-on-chip microcontrollers such as the ESP32, as well as low-cost capacitive soil-moisture sensors and Hall-effect turbine flow sensors, a small group of people can now create a prototype of a closed-loop irrigation controller that is both locally autonomous and aware of the weather without having to constantly connect to a cloud-based backend for its primary safety features.

Drip irrigation delivers water at a low flow rate directly to the root zone of a crop and is widely promoted as a more water-efficient alternative to flood irrigation. However, the efficiency of a drip system depends not only on the delivery method but also on when, and for how long, water is applied. Automated controllers that close the loop between a soil-moisture reading and a pump or valve are well established [1], [3], [5], and most of them operate on a simple threshold: irrigation starts when the measured soil moisture falls below a lower bound and stops when it rises above an upper bound. This reactive strategy is simple and robust, but it has a blind spot. It cannot distinguish between soil that is dry and will remain dry and soil that is dry but is about to receive rain. A controller that starts the pump shortly before a forecast rainfall event spends water and pumping energy on moisture that the weather would have supplied at no cost, and in the worst case can over-wet the root zone.

Predictive and precision scheduling approaches have been explored in the literature, ranging from platforms that fuse several sensor streams with external data sources across pilot sites [2], to distributed edge, fog and cloud architectures [6], to scheduling methods based on reference evapotranspiration [13]. These approaches are valuable, but they generally assume a continuously connected back-end and a level of infrastructure and engineering effort that is difficult for a small team, or a smallholder farm, to reproduce. Surveys of the use of IoT in agriculture also point to a persistent gap between laboratory prototypes and field-validated deployments [19], [20]. There is therefore room for a simple, inexpensive and locally autonomous design in which the predictive element is limited to a single, easily understood decision.

This paper presents the development of such a prototype. A key design choice was to make the predictive decision — whether to delay irrigation based on a weather forecast — locally on the ESP32 via a direct HTTP request to a public weather API, instead of making the decision remotely via a model whose availability would be needed for the irrigation

safety logic to work. In the prototype, irrigation is withheld only when the soil is below its dry threshold *and* the forecast probability of precipitation exceeds a configurable threshold. If the WiFi connection or the weather API is unavailable, the controller falls back to reactive-only behaviour rather than blocking irrigation.

Additionally, a second, separate layer of logic implements two hardware-related safety interlocks (maximum run-time, and a no-flow fault using the same pulse count used for volumetric accounting) that will work regardless of whether the predictive logic, or even the network connection, is currently working. The reason for keeping the safety logic independent of the network is practical: a pump that is switched by a controller whose safety behaviour depends on a remote service can be left running if that service or the connection becomes unavailable. In the prototype, the maximum run-time limit bounds the duration of any single irrigation event, and the no-flow fault forces the pump off if the pump has been commanded on but no flow pulses are detected within a grace period, for example because of a dry-running pump, a closed valve or a blocked line.

The Hall-effect turbine flow sensor serves a second purpose. By counting pulses and dividing the count by a calibration constant, the prototype computes the volume of water delivered during each irrigation event (Section VI), which gives a basis for water-consumption monitoring that does not depend on estimating the volume from the pump run-time. The accuracy of this volume, and of the soil-moisture percentage derived from the capacitive sensor, depends on per-unit calibration [8], [9]. In the firmware these calibration constants are still placeholder values, and this paper therefore treats the volumetric and moisture outputs as uncalibrated.

A second contribution of this work is methodological. During the project, a large amount of documentation was produced, including design documents, an invention disclosure and presentation decks. Several of these documents contained claims (e.g., a specific flow-sensor model, a verified relay polarity, an audible-alert subsystem) that could not be confirmed from independent sources once the documents were cross-referenced. Rather than carrying these claims into this paper, each hardware and software claim is classified by its supporting evidence (Section VII), and the paper lists what has been implemented, what has been observed, and what remains open (Section IX).

This paper does not report field trials, measured water savings, calibration accuracy, forecast-skip precision or confirmed relay polarity. The prototype has been assembled and its sensing and serial-telemetry path has been observed running on the bench, but the remaining items are outstanding work and are neither inferred nor simulated. Cost-effective, field-deployable evaluation of such systems is comparatively rare in the literature [4], and reporting implementation status accurately was preferred to presenting simulated figures.

The contributions of this paper are as follows:

1) A locally resolved predictive-skip irrigation controller for the ESP32, combining a soil-moisture threshold with a forecast probability-of-precipitation check, with a fallback to reactive-only operation when the forecast is unavailable.
2) Two independent safety interlocks, a maximum pump run-time and a no-flow fault, that operate regardless of the predictive logic or the network state.
3) A pulse-count volumetric accounting model (Eq. 1) that supports water-consumption monitoring using a Hall-effect flow sensor.
4) An evidentiary classification applied to every hardware and software claim, which separates what is implemented, what is physically observed and what remains unverified.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the system architecture and separates the implemented local chain from the proposed cloud extension. Section IV presents the hardware design, Section V the firmware and control logic, and Section VI the water-balance model. Section VII sets out the evidentiary classification, Section VIII reports the bench observations, Section IX discusses limitations and threats to validity, and Section X concludes with future work.

II. Related Work

A. Closed-loop and sensor-network irrigation controllers

Wireless-sensor-network (WSN) irrigation controllers that connect a soil-moisture measurement to an actuator are well established. Gutiérrez *et al.* [1] developed an automated drip-irrigation system around a WSN and a GPRS module, with a threshold-based local decision and remote monitoring, and reported measured water savings compared with a manually scheduled control plot. The design is representative of the reactive approach on which many later systems build: a simple threshold on the current soil state, a low-power sensing network, and a remote link for monitoring. Vaishali *et al.* [3] describe a mobile-integrated IoT irrigation system that similarly combines local sensing with a companion mobile application for status and control. In both cases the irrigation decision depends only on the present measurement; neither uses a forecast to anticipate rainfall, which is the gap the controller in this paper addresses.

B. Predictive and precision scheduling

Some work goes beyond reactive thresholding toward predictive or precision scheduling. Kamienski *et al.* [2] developed SWAMP, a smart-water-management platform that integrates several sensor streams with external data sources for precision irrigation at multiple pilot sites, at a scale much larger than a single-node bench prototype. Ferrández-Pastor *et al.* [6] proposed a distributed edge/fog/cloud computing architecture for precision agriculture, which is architecturally similar to the local/cloud split described in this project's design documents; however, as discussed in Section III, only the local half of that split is implemented in the firmware. Platform-level systems of this kind offer considerable capability, but they assume back-end services, data management and engineering effort that

are difficult for a small team or a single farm to reproduce, and their irrigation behaviour depends on those services being available.

A different route to predictive scheduling is the use of reference evapotranspiration. The FAO Penman–Monteith method described in FAO Irrigation and Drainage Paper 56 [13] estimates crop water requirements from weather variables and crop coefficients, and is a widely used basis for irrigation scheduling. It does not require a flow sensor, but it does require weather inputs and crop-specific parameters, and it estimates water demand rather than measuring the water actually delivered. The prototype in this paper takes the complementary approach of measuring delivered volume directly with a Hall-effect flow sensor, and uses the weather forecast only for a single binary decision: whether to withhold irrigation because rain is likely. That forecast comes from a public forecast service [15] through a direct request from the ESP32, so no intermediate back-end is involved.

C. Connectivity choices

Connectivity is a further design variable. Križanović *et al.* [7] reported a LoRa-based WSN for precision-agriculture environmental monitoring, demonstrating a longer-range and energy-efficient alternative to the direct WiFi link used by this prototype between the edge controller and the weather API. A LoRa network is well suited to monitoring spread across a large area, but reaching an internet forecast service from it requires a gateway. WiFi is a reasonable choice for a single controller within range of an access point, which is the setting of the bench prototype described here, and it allows the weather request to be made directly from the microcontroller.

D. Surveys of agricultural IoT

Surveys place these systems in a wider context. Jha *et al.* [4] surveyed the use of artificial intelligence in agricultural automation, including irrigation scheduling, and noted that cost-effective, field-deployable evaluation of such systems, as opposed to simulation-only evaluation, is relatively rare in the literature. This observation is the reason this paper reports implementation status rather than simulated results. García *et al.* [5] surveyed the sensing hardware and system architectures used across published IoT-based smart-irrigation systems; the hardware selected for this project (capacitive soil sensor, Hall-effect flow sensor, relay-driven actuation) is broadly consistent with the designs they describe. At the wider agricultural-IoT level, Tzounis *et al.* [19] and Verdouw *et al.* [20] separately surveyed IoT adoption in agriculture, and both identify a gap between lab-scale IoT prototypes and field-validated deployments. This paper addresses that gap only for a single bench prototype, by stating the validation status of each claim explicitly, and does not attempt to close it for the field as a whole.

E. Calibration of low-cost sensors

Low-cost capacitive soil-moisture sensors, such as the one used here, produce a raw output that must be converted to a moisture value through calibration. Adla *et al.* [9] report laboratory calibration and a performance evaluation of low-cost capacitive and very low-cost resistive soil-moisture sensors, and Abdelmoneim *et al.* [8] describe the calibration of low-cost capacitive sensors for irrigation-management applications. These studies motivate the calibration step that is still outstanding for this prototype (Section V): until the sensor and the flow meter are calibrated, the moisture percentage and delivered volume the firmware reports should be read as uncalibrated values.

F. Positioning of this work

This project differs from the work above in scope rather than in the novelty of individual components. It does not claim a new sensing principle or a new machine-learning model. Instead, it evaluates how far a single inexpensive edge controller can go in combining reactive and predictive irrigation logic, two independent safety interlocks, and volumetric accounting from a pulse-counting flow sensor, while stating clearly what has and has not yet been bench-validated.

III. System Architecture

Figure 1 separates the system into the chain implemented in the firmware (`irrigation_prototype_predictive.ino`) and a cloud-connected extension (MQTT communication, a FastAPI backend, a PostgreSQL database, a sensor-trend predictive model, and a companion mobile application) that is described in the project's design documents and invention disclosure but is not implemented in the firmware. The data path the firmware actually uses is simple: a direct, unencrypted HTTP GET from the ESP32 to the OpenWeatherMap 5-day/3-hour forecast endpoint [15], consumed locally with no round trip through any backend service.

The implemented chain operates as a single fast loop on the ESP32: read sensors, evaluate the local safety interlocks, evaluate the irrigation decision (reactive threshold plus predictive weather skip), drive the relay, and print a telemetry line to the USB serial console. No part of this loop depends on network connectivity except the predictive-skip feature itself, which fails open to reactive-only behaviour when WiFi or the weather API is unavailable (Section V).

IV. Hardware Design

The edge controller is an ESP32-WROOM-32 development board [14], chosen for its dual ADC/interrupt-capable GPIO, built-in WiFi, and low unit cost. Table I lists the components documented in the project's design report and build guide. The last column states whether each component's presence in the assembled prototype is supported by a photograph (Category 4 in the evidence scheme of Section VII) or only by design documentation (Category 1).

Figure 2 shows the pin-level connections as specified by the firmware's pin definitions and header comments. It is a design schematic derived from the source code and build documentation, not a traced photograph of the assembled

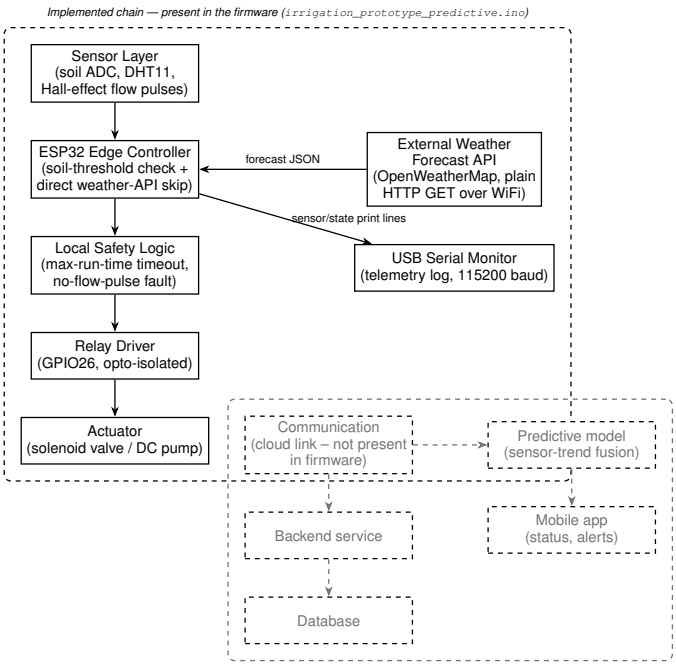


Fig. 1. System architecture, with the implemented local chain separated from the proposed (not-yet-implemented) cloud extension described in the project's design documents.

TABLE I
HARDWARE BILL OF MATERIALS AND EVIDENTIARY BASIS

Component	Role	Confirmed by photo?
ESP32-WROOM-32 DevKit	Edge controller: acquisition, local decision, actuation	Yes
Capacitive soil-moisture sensor (labelled "V2.0.0")	Soil-moisture input, GPIO34 (ADC1)	Yes
DHT11 temperature/humidity sensor	Ambient temperature and humidity input, GPIO4	Probable (blue module consistent with a DHT11; label not legible)
Hall-effect turbine flow sensor	Pulse counting for volumetric accounting, GPIO27	Yes (exact part number not legible)
1-channel opto-isolated relay module	Driver stage, GPIO26	Yes
Solenoid valve / small DC pump	Water-delivery actuator	Pump: yes (model not legible); valve: not visible
Separate DC supply, flyback diode, fuse	Actuator power and protection	Design document only
Audible alert (buzzer)	Optional status indication	Probable (round component consistent with a buzzer); pin, wiring and firmware not confirmed

board. The relay polarity, i.e. whether GPIO26 HIGH or LOW energises the relay coil on the module used, has not been confirmed and is marked as such on the diagram.

The prototype was assembled on a breadboard according to the schematic in Fig. 2, using the components listed in Table I. Photographs of the assembled hardware (not reproduced here)

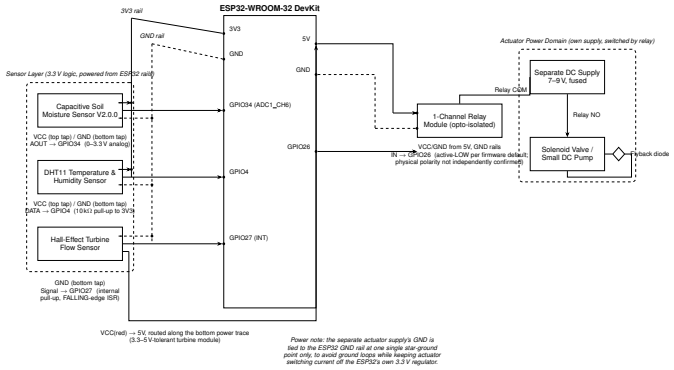


Fig. 2. Pin-level circuit schematic derived from the firmware and build documentation. This is a design diagram, not a photograph of the assembled hardware.

are consistent with this configuration.

V. Firmware and Control Logic

The firmware (`irrigation_prototype_predictive.ino`) is written against the Arduino-ESP32 core [18] and depends on two third-party libraries declared in its own header comments: the Adafruit DHT sensor library [16] for the temperature/humidity sensor, and ArduinoJson [17] for parsing the weather-API response. It implements the control flow in Figure 3. Four elements are described precisely below.

1) **Soil moisture** is read as a raw ADC value on GPIO34 and linearly mapped to a 0–100% scale using two calibration constants, `SOIL_DRY_VALUE` and `SOIL_WET_VALUE`, which the firmware's own comments mark as placeholders to be replaced with values measured on the specific sensor unit used. Published studies of low-cost capacitive soil-moisture sensors [8], [9] describe calibrating the sensor output against reference moisture measurements, which is why a per-unit two-point calibration, not yet performed on this prototype's sensor, is needed before `soilPct` can be treated as an accurate percentage rather than a relative indicator.
2) **Volumetric flow** is computed by dividing the Hall-effect sensor's pulse count by a calibration constant, `FLOW_CALIBRATION_PULSES_PER_L`, which is likewise marked as a placeholder (default value 450.0 pulses/L) pending a measured calibration against a known poured volume (Section VI).
3) **The predictive skip** fires only when `soilPct` is below the dry threshold *and* the most recent OpenWeatherMap forecast's probability-of-precipitation field exceeds a configurable threshold (default 0.40); if the WiFi connection or the API call has never succeeded, the system falls back to reactive-only behaviour by design, rather than blocking irrigation on an unavailable forecast.
4) **Relay polarity** is controlled by a single boolean, `RELAY_ACTIVE_LOW`, set to `true` by default; the

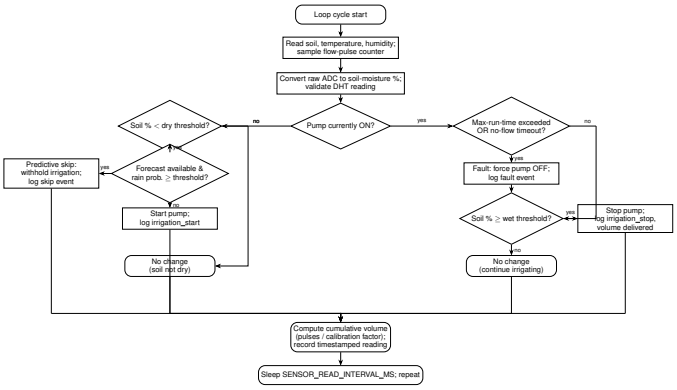


Fig. 3. Control flow of the main loop, reconstructed from the firmware.

Algorithm 1 Irrigation decision (as implemented)

if pump is OFF **then**
  if soilPct $<$ SOIL_DRY_THRESHOLD_PCT **then**
    if weatherAvailable **and** rainExpected **then**
      log "predictive skip"; leave pump OFF
    else
      turn pump ON
    end if
  end if
else
  if soilPct $\geq$ SOIL_WET_THRESHOLD_PCT **then**
    turn pump OFF
  end if
end if
{evaluated every loop, independent of the above:}
if pump is ON **and** (run-time $>$ MAX_PUMP_RUN_MS **or** no new pulses for $>$ NO_FLOW_GRACE_MS) **then**
  force pump OFF; log fault
end if

source comment directly above it reads "VERIFY WITH YOUR OWN MODULE", i.e. the firmware's own author flagged this value as unverified at the time of writing. It is therefore treated here as an open item rather than a tested result.

Algorithm 1 restates the irrigation-decision function in pseudocode, matching the source in its conditional structure.

VI. Water-Balance Model

The Hall-effect turbine sensor produces output pulses at a rate approximately proportional to the instantaneous volumetric flow rate Q, so that the delivered volume over an irrigation event is

$$V = \frac{N}{K} \tag{1}$$

where N is the pulse count accumulated during the event (read from the ESP32 interrupt-driven counter on GPIO27, falling-edge triggered) and K is an empirically-measured calibration constant, in pulses per litre, obtained by passing a known volume through the sensor and dividing the resulting pulse count by that volume. This is the formula the firmware implements (`pulsesToLitres()`). An earlier reference formula used during the design phase was dimensionally inconsistent and was corrected before any hardware was built. Because the calibration constant K in the firmware is still the placeholder value (450.0 pulses/L) rather than a value measured on the specific sensor unit assembled, any volume figure the current build would report should be treated as uncalibrated until that measurement step is carried out and the constant is updated accordingly. This direct, pulse-counted volumetric model is a deliberately simpler alternative to standard reference-evapotranspiration-based scheduling methods such as the FAO Penman–Monteith method [13], which estimate crop water demand from weather variables rather than measuring delivered volume directly; the firmware does not implement an evapotranspiration model of any kind.

TABLE II
Evidentiary basis of the main claims in this paper

Claim	Category
Pin map (GPIO34/4/27/26) and control logic described in Sections V–VI	4 (firmware source file)
Physical assembly of ESP32, soil sensor, flow sensor, relay and pump on a breadboard	4 (photograph)
Live sensor telemetry (temperature, humidity, soil raw value, zero flow pulses, relay OFF) on 1 Sept. 2026	4 (photograph of serial monitor)
Relay energises on GPIO26 LOW (active-LOW)	5 — unverified; firmware default only
Flow sensor is a specific YF-S201 or YF-S401 model	5 — unverified; generic Hall-effect turbine type confirmed, exact model not legible in available photo
A buzzer/audible alert was implemented on a specific GPIO pin	5 — a `beep()` function was visible in one photographed code screenshot of a differently named firmware file; the pin assignment, and whether this version was ever flashed, are unconfirmed
Physical hardware assembly was carried out by the student team	2 (team-confirmed)
Water savings, prediction accuracy, calibration accuracy	5 — no quantitative test has been performed

VII. Evidentiary Classification

The project's supporting materials include both independently sourced documents and content drafted during earlier design phases. Every substantive claim in this paper is therefore classified into one of five categories: (1) design-phase information or code that has not been independently verified; (2) implementation steps confirmed by the student team; (3) outputs or observations recorded in project notes but not reproducible from a file; (4) primary evidence (a source file or photograph); and (5) claims that remain unverified. Table II applies this scheme to the main claims made, or declined, in this paper.

VIII. Bench Observations

The only run-time evidence available is a single photograph of a laptop screen, dated 1 September 2026, 10:29 PM, showing the Arduino IDE serial monitor connected to an ESP32 over `COM7` at 115200 baud. The logged lines show temperature steady at 22.90°C, humidity in the range 63.2–63.3%, a soil reading reported as 100% (raw ADC values 1171–1196, consistent with the sensor being in a fully wet reference condition), flow pulses at 0 throughout, the forecast-skip variable reporting "Rain expected: NO", and the relay reporting "OFF" on every line. This is consistent with a bench test under no active water flow, as also described in the project's second review presentation. It demonstrates that the sensor-acquisition and serial-telemetry path of some version of the firmware executed on real hardware and produced live, internally consistent multi-sensor readings; it does *not* demonstrate pump actuation, relay polarity, flow-sensor accuracy, or any outcome of the predictive-skip logic under an actual rain event, none of which appear in this or any other available material. The firmware file name visible in that screenshot (`irrigation_predictive_combined.ino`) differs from the file described in Sections V–VI (`irrigation_prototype_predictive.ino`); the two share the same pin map and the same active-low ternary pattern in their relay-driving function, but are not byte-identical, and the exact contents of the version actually running in the photograph are not otherwise available for inspection.

IX. Limitations and Threats to Validity

This section lists the limits of the evidence presented in this paper.

- **No calibration has been performed.** Both calibration constants used by the firmware (`SOIL_DRY_VALUE`/`SOIL_WET_VALUE` and `FLOW_CALIBRATION_PULSES_PER_L`) remain at their placeholder defaults in the firmware source file. Any moisture percentage or volume figure the current build would output should be treated as uncalibrated.
- **Relay polarity is unverified.** The firmware defaults to active-LOW and flags this for verification; an earlier project design diagram assumed the opposite polarity. Neither has been checked against the physical relay module, and this paper does not assert which is correct.
- **The flow-sensor model number is unconfirmed.** The sensor visible in the photograph is a small Hall-effect turbine type consistent with the YF-S* sensor family used in the project's build guide, but its printed label is not legible at the resolution available, and two different model numbers appear across the project's own documents.
- **Buzzer/alert hardware is not confirmed as implemented or tested.** A `beep()` function is visible in one photographed code screenshot, and a component consistent with a buzzer appears in the bench photograph, but the function belongs to a firmware file whose full contents are not available, so neither the GPIO assignment, nor the wiring, nor whether the alert was ever exercised on hardware can be stated with confidence.
- **No water-balance, prediction-accuracy, or water-saving results exist.** The project's own design report explicitly places any claim of measured water savings, sensor accuracy, or predictive-model performance out of scope for the design phase it documents, and no later material supplies such a measurement. This paper accordingly reports implementation status rather than experimental results.
- **Single-sample bench evidence.** The one serial-monitor observation in Section VIII is a single time point under static, no-flow conditions and should not be read as a repeated or statistically meaningful test.

The additional tests that would be needed to move the corresponding claims from "unverified" to "verified" are, respectively: a multimeter or oscilloscope check of the relay coil's drive state at each GPIO26 level; a legible photograph or purchase record of the flow sensor's printed label; the source file and a hardware test of whichever firmware version actually contains the `beep()` function; and a controlled calibration procedure (known-volume pours for the flow sensor, open-air/submerged readings for the soil sensor) followed by an irrigation run with before/after soil-moisture logging.

X. Conclusion and Future Work

This paper has presented the design and current bench-implementation status of an ESP32-based predictive drip-irrigation controller that combines local soil-moisture thresholding with a direct weather-forecast check and two independent safety interlocks. The evidence classification in Table II separates design diagrams, observed behaviour and unverified claims, and can be reused in other early-stage hardware projects whose documentation outpaces their verified results.

Future work, in priority order, is: (i) perform and record the two outstanding sensor calibrations; (ii) confirm relay polarity against the physical module and correct whichever of the firmware default or the design diagram is wrong; (iii) run a controlled irrigation cycle with before/after soil-moisture and delivered-volume logging to produce the first quantitative results; and (iv) only once local behaviour is validated, implement the proposed cloud extension (Fig. 1, lower half).

Acknowledgment

The authors thank the project supervisor for guidance during the design phase of the project.

References

[1] J. Gutiérrez, J. F. Villa-Medina, A. Nieto-Garibay, and M. Á. Porta-Gándara, "Automated irrigation system using a wireless sensor network and GPRS module," *IEEE Trans. Instrum. Meas.*, vol. 63, no. 1, pp. 166–176, Jan. 2014, doi: 10.1109/TIM.2013.2276487.

[2] C. Kamienski *et al.*, "Smart water management platform: IoT-based precision irrigation for agriculture," *Sensors*, vol. 19, no. 2, p. 276, 2019, doi: 10.3390/s19020276.

[3] S. Vaishali, S. Suraj, G. Vignesh, S. Dhivya, and S. Udhayakumar, "Mobile integrated smart irrigation management and monitoring system using IOT," in *Proc. 2017 Int. Conf. Commun. Signal Process. (ICCSP)*, 2017, doi: 10.1109/ICCSP.2017.8286792.

[4] K. Jha, A. Doshi, P. Patel, and M. Shah, "A comprehensive review on automation in agriculture using artificial intelligence," *Artif. Intell. Agric.*, vol. 2, pp. 1–12, 2019, doi: 10.1016/j.aiia.2019.05.004.

[5] L. García, L. Parra, J. M. Jimenez, J. Lloret, and P. Lorenz, "IoT-based smart irrigation systems: An overview on the recent trends on sensors and IoT systems for irrigation in precision agriculture," *Sensors*, vol. 20, no. 4, p. 1042, 2020, doi: 10.3390/s20041042.

[6] F. J. Ferrández-Pastor, J. M. García-Chamizo, M. Nieto-Hidalgo, and J. Mora-Martínez, "Precision agriculture design method using a distributed computing architecture on internet of things context," *Sensors*, vol. 18, no. 6, p. 1731, 2018, doi: 10.3390/s18061731.

[7] V. Križanović, K. Grgić, J. Spišić, and D. Žagar, "An advanced energy-efficient environmental monitoring in precision agriculture using LoRa-based wireless sensor networks," *Sensors*, vol. 23, no. 14, p. 6332, 2023, doi: 10.3390/s23146332.

[8] A. A. Abdelmoneim, C. M. Al Kalaany, R. Khadra, B. Derardja, and G. Dragonetti, "Calibration of low-cost capacitive soil moisture sensors for irrigation management applications," *Sensors*, vol. 25, no. 2, p. 343, 2025, doi: 10.3390/s25020343.

[9] S. Adla, N. K. Rai, S. H. Karumanchi, S. Tripathi, M. Disse, and S. Pande, "Laboratory calibration and performance evaluation of low-cost capacitive and very low-cost resistive soil moisture sensors," *Sensors*, vol. 20, no. 2, p. 363, 2020, doi: 10.3390/s20020363.

[10] NITI Aayog, Government of India, "Composite water management index 2.0," NITI Aayog, New Delhi, India, Tech. Rep., 2019. [Online]. Available: https://www.niti.gov.in/sites/default/files/2023-03/Composite%20Water%20Management%20Index%202.0.pdf

[11] Food and Agriculture Organization of the United Nations, "AQUASTAT country profile — India," FAO AQUASTAT Global Information System on Water and Agriculture, 2024. [Online]. Available: https://www.fao.org/aquastat/en/geospatial-information/global-maps-irrigated-areas/irrigation-by-country/country/IND

[12] M. Rodell, I. Velicogna, and J. S. Famiglietti, "Satellite-based estimates of groundwater depletion in India," *Nature*, vol. 460, no. 7258, pp. 999–1002, Aug. 2009, doi: 10.1038/nature08238.

[13] R. G. Allen, L. S. Pereira, D. Raes, and M. Smith, *Crop Evapotranspiration — Guidelines for Computing Crop Water Requirements*, FAO Irrigation and Drainage Paper 56. Rome, Italy: Food and Agriculture Organization of the United Nations, 1998.

[14] Espressif Systems, *ESP32 Technical Reference Manual*, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

[15] OpenWeather Ltd., "5 day / 3 hour forecast API documentation," OpenWeatherMap, 2024. [Online]. Available: https://openweathermap.org/forecast5

[16] Adafruit Industries, "DHT sensor library (Arduino)," GitHub repository, 2024. [Online]. Available: https://github.com/adafruit/DHT-sensor-library

[17] B. Blanchon, "ArduinoJson," 2024. [Online]. Available: https://arduinojson.org

[18] Espressif Systems, "Arduino core for the ESP32," GitHub repository, 2024. [Online]. Available: https://github.com/espressif/arduino-esp32

[19] A. Tzounis, N. Katsoulas, T. Bartzanas, and C. Kittas, "Internet of Things in agriculture, recent advances and future challenges," *Biosyst. Eng.*, vol. 164, pp. 31–48, Dec. 2017.

[20] C. Verdouw, S. Wolfert, and B. Tekinerdogan, "Internet of Things in agriculture," *CAB Rev.*, vol. 11, no. 035, pp. 1–12, 2016, doi: 10.1079/PAVSNNR201611035.